```
import os
import asyncio
import logging
from datetime import datetime, timedelta
from flask import Flask
from threading import Thread
from pyrogram import Client, filters, idle
from pyrogram.types import
InlineKeyboardMarkup, InlineKeyboardButton,
Message, CallbackQuery
from pyrogram.errors import (
    ChannelInvalid, ChannelPrivate,
UsernameNotOccupied,
    UserNotParticipant, FloodWait,
ChatAdminRequired, PeerIdInvalid
)
from pymongo import MongoClient
from pymongo.errors import
ConnectionFailure

logging.basicConfig(level=logging.INFO,
format="%(asctime)s [%(levelname)s] %
(message)s")
log = logging.getLogger(__name__)

API_ID = int(os.environ.get("API_ID", "0"))
API_HASH = os.environ.get("API_HASH", "")
BOT_TOKEN = os.environ.get("BOT_TOKEN", "")
MONGO_URL = os.environ.get("MONGO_URL", "")
AUTH_CHANNEL =
```

```
os.environ.get("AUTH_CHANNEL",
"MOTIVATINALTHOUGHTS")
ADMIN_IDS = [int(x) for x in
os.environ.get("ADMIN_IDS", "0").split(",")
if x.strip().isdigit()]
ADMIN_USERNAME =
os.environ.get("ADMIN_USERNAME",
"YourAdminUsername")

FREE_LIMIT = 3
PREMIUM_LIMIT = 1000
MONITOR_INTERVAL = 3600

try:
    mongo = MongoClient(MONGO_URL,
serverSelectionTimeoutMS=5000)
    mongo.server_info()
    db = mongo["channel_guardian_db"]
    log.info("MongoDB connected!")
except Exception as e:
    log.error(f"MongoDB error: {e}")
    db = None

users_col = db["users"] if db is not None
else None
channels_col = db["channels"] if db is not
None else None
messages_col = db["messages"] if db is not
None else None

bot = Client(
    "channel_guardian",
    api_id=API_ID,
    api_hash=API_HASH,
```

```
    bot_token=BOT_TOKEN,
    in_memory=True
)


def get_user(user_id):
    return users_col.find_one({"user_id":
user_id})


def upsert_user(user_id, data):
    users_col.update_one(
        {"user_id": user_id},
        {"$set": data, "$setOnInsert":
{"joined": datetime.now(), "user_id":
user_id}},
        upsert=True
    )


def is_premium(user_id):
    user = get_user(user_id)
    if not user:
        return False
    until = user.get("premium_until")
    return bool(until and until >
datetime.now())


def get_limit(user_id):
    return PREMIUM_LIMIT if
is_premium(user_id) else FREE_LIMIT
```

```
def get_channels(user_id):
    return
list(channels_col.find({"user_id":
user_id}))


def get_channel(user_id, username):
    return
channels_col.find_one({"user_id": user_id,
"username": username.lower()})


async def check_joined(client, message):
    if not AUTH_CHANNEL:
        return True
    try:
        await
client.get_chat_member(AUTH_CHANNEL,
message.from_user.id)
        return True
    except UserNotParticipant:
        btn = [[InlineKeyboardButton("📢
Join Channel",
url=f"https://t.me/{AUTH_CHANNEL}")]]
        await message.reply(
            "🔒 **Pehle hamare channel ko
join karo!**\n\nJoin karne ke baad dobara
command use karo.",

reply_markup=InlineKeyboardMarkup(btn)
        )
        return False
    except Exception:
        return True
```

```
async def fetch_channel_info(client,
username):
    try:
        chat = await
client.get_chat(username)
        return {
            "alive": True,
            "title": chat.title,
            "members": getattr(chat,
"members_count", 0),
            "username": getattr(chat,
"username", None),
            "invite_link": getattr(chat,
"invite_link", None),
            "description": getattr(chat,
"description", ""),
        }
    except (ChannelInvalid, ChannelPrivate,
ChatAdminRequired):
        return {"alive": False, "reason":
"banned"}
    except (UsernameNotOccupied,
PeerIdInvalid):
        return {"alive": False, "reason":
"deleted"}
    except Exception as e:
        return {"alive": False, "reason":
"error", "detail": str(e)}


def home_text(name, user_id):
    premium = is_premium(user_id)
```

```
    badge = "💎 Premium" if premium else
"🆓 Free"
    limit = get_limit(user_id)
    count = len(get_channels(user_id))
    if premium:
        user = get_user(user_id)
        until =
user["premium_until"].strftime("%d %b %Y")
        badge += f" (till {until})"
    return (
        f"👋 **Welcome, {name}!**\n\n"
        f"🛡️ **Channel Guardian Bot**\n"
        f"Aapke channels ko 24/7 silently
monitor karta hoon.\n"
        f"Ban hone par **turant alert**
bhejta hoon.\n\n"
        f"━━━━━━━━━━━━━━━━━\n"
        f"**📊 Plan:** {badge}\n"
        f"**📡 Channels:**
{count}/{limit}\n"
        f"━━━━━━━━━━━━━━━━━\n\n"
        f"Niche se koi bhi option choose
karo 👇"
    )


def home_buttons(user_id):
    premium = is_premium(user_id)
    rows = [
        [
            InlineKeyboardButton("📋 My
Channels", callback_data="my_channels"),
            InlineKeyboardButton("➕ Add
Channel", callback_data="how_to_add"),
```

```
        ],
        [
            InlineKeyboardButton("💎
Premium" if not premium else "💎 My Plan",
callback_data="premium_menu"),
            InlineKeyboardButton("📖 Help",
callback_data="help_menu"),
        ],
        [
            InlineKeyboardButton("📢
Official Channel",
url=f"https://t.me/{AUTH_CHANNEL}"),
        ]
    ]
    return InlineKeyboardMarkup(rows)


@bot.on_message(filters.command("start"))
async def start_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    user_name =
message.from_user.first_name
    upsert_user(user_id, {
        "name": user_name,
        "username":
message.from_user.username,
        "last_seen": datetime.now(),
    })
    await
message.reply(home_text(user_name,
user_id),
```

```
reply_markup=home_buttons(user_id))


@bot.on_message(filters.command("add"))
async def add_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    args = message.text.split()
    if len(args) < 2:
        await message.reply(
            "❌ **Usage:** `/add
@username`\n\n"
            "**Example:** `/add
@mychannel`"
        )
        return
    username = args[1].replace("@",
"").strip().lower()
    limit = get_limit(user_id)
    channels = get_channels(user_id)
    if len(channels) >= limit:
        if not is_premium(user_id):
            btn =
[[InlineKeyboardButton("💎 Premium Lo",
callback_data="premium_menu")]]
            await message.reply(
                f"❌ **Free limit full!
({FREE_LIMIT}/{FREE_LIMIT})**\n\n"
                f"💎 **Premium** lo aur
**1000+ channels** monitor karo sirf
**₹99/month** mein!",
```

```
reply_markup=InlineKeyboardMarkup(btn)
            )
        else:
            await message.reply(f"❌
Maximum limit ({PREMIUM_LIMIT}) reach ho
gayi.")
        return
    if get_channel(user_id, username):
        await message.reply(f"⚠️
`@{username}` pehle se added hai!")
        return
    msg = await message.reply(f"🔍
`@{username}` check kar raha hoon...")
    info = await fetch_channel_info(client,
username)
    if info["alive"]:
        channels_col.insert_one({
            "user_id": user_id,
            "username": username,
            "title": info["title"],
            "members": info["members"],
            "added_on": datetime.now(),
            "last_checked": datetime.now(),
            "last_status": "active",
            "invite_link":
info.get("invite_link"),
            "keywords": [],
            "notify": True,
        })
        await msg.edit(
            f"✅ **Channel Added!**\n\n"
            f"📛 **Name:**
{info['title']}\n"
            f"🔗 **Username:**
```

```
@{username}\n"
            f"👥 **Members:**
{info['members']:,}\n\n"
            f"🛡️ Ab main isko **24/7
monitor** karta rahunga!"
        )
    elif info["reason"] == "banned":
        await msg.edit(f"❌ `@{username}`
already banned/private hai.")
    elif info["reason"] == "deleted":
        await msg.edit(f"❌ `@{username}`
exist nahi karta.")
    else:
        await msg.edit(f"⚠️ Error:
`{info.get('detail', 'Unknown')}`")


@bot.on_message(filters.command("list"))
async def list_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    channels = get_channels(user_id)
    if not channels:
        await message.reply(
            "📭 **Koi channel add nahi kiya
abhi tak.**\n\n"
            "Use `/add @username` to start
monitoring!"
        )
        return
    limit = get_limit(user_id)
    text = f"📋 **Monitored Channels
```

```
({len(channels)}/{limit}):**\n\n"
    for i, ch in enumerate(channels, 1):
        emoji = "✅" if
ch.get("last_status") == "active" else "❌"
        title = ch.get("title",
ch["username"])
        added = ch["added_on"].strftime("%d
%b %Y")
        checked = ch.get("last_checked")
        checked_str = checked.strftime("%d
%b, %I:%M %p") if checked else "Never"
        kw_count = len(ch.get("keywords",
[]))
        text += (
            f"{i}. {emoji} **{title}**\n"
            f"   🔗 @{ch['username']}\n"
            f"   📅 Added: {added}\n"
            f"   🕐 Last check:
{checked_str}\n"
        )
        if kw_count:
            text += f"   🔑 Keywords:
{kw_count} set\n"
        text += "\n"
    await message.reply(text)


@bot.on_message(filters.command("remove"))
async def remove_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    args = message.text.split()
```

```
    if len(args) < 2:
        await message.reply("❌ **Usage:** `/remove @username`")
        return
    username = args[1].replace("@", "").strip().lower()
    result = channels_col.delete_one({"user_id": user_id, "username": username})
    if result.deleted_count:
        await message.reply(f"✅ `@{username}` monitoring se hata diya gaya!")
    else:
        await message.reply(f"❌ `@{username}` aapki list mein nahi tha.")


@bot.on_message(filters.command("status"))
async def status_cmd(client, message):
    if not await check_joined(client, message):
        return
    args = message.text.split()
    if len(args) < 2:
        await message.reply("❌ **Usage:** `/status @username`")
        return
    username = args[1].replace("@", "").strip()
    msg = await message.reply(f"🔍 Checking `@{username}`...")
    info = await fetch_channel_info(client, username)
```

```
    if info["alive"]:
        await msg.edit(
            f"✅ **Channel Active
Hai!**\n\n"
            f"📛 **Name:**
{info['title']}\n"
            f"🔗 **Username:**
@{username}\n"
            f"👥 **Members:**
{info['members']:,}"
        )
    elif info["reason"] == "banned":
        await msg.edit(f"🚨 **`@{username}`
BAN HO GAYA HAI!**\n\nChannel ab accessible
nahi hai.")
    elif info["reason"] == "deleted":
        await msg.edit(f"🗑️ **`@{username}`
delete ho gaya hai.**")
    else:
        await msg.edit(f"⚠️ Error:
`{info.get('detail')}`")


@bot.on_message(filters.command("keywords")
)
async def keywords_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    parts = message.text.split(None, 2)
    if len(parts) < 3:
        await message.reply(
            "🔑 **Keyword Alert
```

```
Setup**\n\n"
            "**Usage:**\n"
            "`/keywords @channel word1,
word2, word3`\n\n"
            "**Example:**\n"
            "`/keywords @mychannel new
link, backup, moved to`\n\n"
            "**Clear karne ke liye:**\n"
            "`/keywords @channel clear`"
        )
        return
    username = parts[1].replace("@",
"").strip().lower()
    kw_input = parts[2].strip()
    ch = get_channel(user_id, username)
    if not ch:
        await message.reply(
            f"❌ `@{username}` list mein
nahi hai.\n"
            f"Pehle `/add @{username}`
karo."
        )
        return
    if kw_input.lower() == "clear":
        channels_col.update_one(
            {"user_id": user_id,
"username": username},
            {"$set": {"keywords": []}}
        )
        await message.reply(f"✅
`@{username}` ke saare keywords clear ho
gaye!")
        return
    keywords = [k.strip().lower() for k in
```

```
kw_input.split(",") if k.strip()]
    channels_col.update_one(
        {"user_id": user_id, "username":
username},
        {"$set": {"keywords": keywords}}
    )
    kw_list = "\n".join([f"  • `{k}`" for k
in keywords])
    await message.reply(
        f"✅ **Keywords set ho gaye
`@{username}` ke liye:**\n\n"
        f"{kw_list}\n\n"
        f"🔔 In words ka koi bhi message
aate hi alert bhejunga!"
    )


@bot.on_message(filters.command("premium"))
async def premium_cmd(client, message):
    if not await check_joined(client,
message):
        return
    user_id = message.from_user.id
    if is_premium(user_id):
        user = get_user(user_id)
        until =
user["premium_until"].strftime("%d %B %Y")
        await message.reply(
            f"💎 **Aap Premium Member
Hain!**\n\n"
            f"📅 **Valid Till:**
{until}\n\n"
            f"✅ 1000+ channels\n"
            f"✅ Real-time instant
```

```
alerts\n"
            f"✅ Message backup\n"
            f"✅ Priority support"
        )
        return
    btn = [[InlineKeyboardButton(f"💳 Buy
Premium — ₹99/month",
url=f"https://t.me/{ADMIN_USERNAME}")]]
    await message.reply(
        "💎 **PREMIUM PLAN —
₹99/month**\n\n"

"┌──────────────────────────────┐\n"
        "│  🆓 Free   vs  💎 Premium │\n"

"├──────────────────────────────┤\n"
        "│ Channels: 3   →  1000+   │\n"
        "│ Alerts:  1hr →  Instant │\n"
        "│ Backup:  ❌   →  ✅       │\n"
        "│ Support: ❌   →  ✅       │\n"

"└──────────────────────────────┘\n\n"
        f"**Contact:**
@{ADMIN_USERNAME}\n\n"
        "Payment ke baad screenshot bhejo —
24hr mein activate!",

reply_markup=InlineKeyboardMarkup(btn)
    )


@bot.on_message(filters.command("help"))
async def help_cmd(client, message):
    if not await check_joined(client,
```

```
message):
        return
    await message.reply(
        "📖 **CHANNEL GUARDIAN — COMPLETE
GUIDE**\n\n"
        "**Commands:**\n"
        "`/add @channel` — Channel monitor
karo\n"
        "`/list` — Sabhi channels dekho\n"
        "`/remove @channel` — Monitoring
band karo\n"
        "`/status @channel` — Abhi check
karo\n"
        "`/keywords @ch w1, w2` — Keywords
set karo\n"
        "`/premium` — Plan upgrade karo\n"
        "`/me` — Apni info dekho\n\n"
        "**How it works:**\n"
        "1️⃣ `/add @channel` karo\n"
        "2️⃣ Bot har ghante silently check
karta hai\n"
        "3️⃣ Ban hone par **turant** alert
aata hai\n"
        "4️⃣ Keywords match hone par bhi
alert aata hai\n\n"
        f"**Issues?** @{ADMIN_USERNAME}"
    )


@bot.on_message(filters.command("me"))
async def me_cmd(client, message):
    if not await check_joined(client,
message):
        return
```

```
    user_id = message.from_user.id
    user = get_user(user_id) or {}
    premium = is_premium(user_id)
    channels = get_channels(user_id)
    limit = get_limit(user_id)
    joined = user.get("joined",
datetime.now()).strftime("%d %b %Y")
    plan_text = "🆓 Free"
    if premium:
        until =
user["premium_until"].strftime("%d %b %Y")
        plan_text = f"💎 Premium (till
{until})"
    await message.reply(
        f"👤 **Your Profile**\n\n"
        f"🆔 **User ID:** `{user_id}`\n"
        f"👤 **Name:**
{message.from_user.first_name}\n"
        f"📅 **Joined:** {joined}\n"
        f"🏷️ **Plan:** {plan_text}\n"
        f"📡 **Channels:**
{len(channels)}/{limit}\n"
    )


@bot.on_message(filters.command("addpremium
") & filters.user(ADMIN_IDS))
async def add_premium_cmd(client, message):
    args = message.text.split()
    if len(args) < 3:
        await message.reply("**Usage:**
`/addpremium USER_ID DAYS`\nExample:
`/addpremium 123456789 30`")
        return
```

```
    try:
        target_id = int(args[1])
        days = int(args[2])
    except ValueError:
        await message.reply("❌ Invalid
format. `/addpremium USER_ID DAYS`")
        return
    until = datetime.now() +
timedelta(days=days)
    users_col.update_one(
        {"user_id": target_id},
        {"$set": {"premium_until": until}},
        upsert=True
    )
    try:
        await bot.send_message(
            target_id,
            f"🎉 **Premium
Activated!**\n\n"
            f"💎 **Plan:** {days} days\n"
            f"📅 **Valid Till:**
{until.strftime('%d %B %Y')}\n\n"
            f"✅ 1000+ channels monitor
karo\n"
            f"✅ Instant alerts\n"
            f"Thank you for supporting us!
🙏"
        )
    except Exception:
        pass
    await message.reply(
        f"✅ User `{target_id}` ko {days}
days premium de diya!\n"
        f"Valid till: {until.strftime('%d
```

```
%b %Y')}"
    )


@bot.on_message(filters.command("removeprem
ium") & filters.user(ADMIN_IDS))
async def remove_premium_cmd(client,
message):
    args = message.text.split()
    if len(args) < 2:
        await message.reply("**Usage:**
`/removepremium USER_ID`")
        return
    try:
        target_id = int(args[1])
    except ValueError:
        await message.reply("❌ Invalid
User ID.")
        return
    users_col.update_one(
        {"user_id": target_id},
        {"$unset": {"premium_until": ""}}
    )
    await message.reply(f"✅ User
`{target_id}` ka premium remove kar diya!")


@bot.on_message(filters.command("broadcast"
) & filters.user(ADMIN_IDS))
async def broadcast_cmd(client, message):
    if not message.reply_to_message:
        await message.reply("❌ Kisi
message ko reply karo aur `/broadcast` use
karo.")
```

```
        return
    msg = await message.reply("📤
Broadcasting...")
    users = list(users_col.find({},
{"user_id": 1}))
    success = 0
    failed = 0
    for user in users:
        try:
            await
message.reply_to_message.forward(user["user
_id"])
            success += 1
            await asyncio.sleep(0.1)
        except Exception:
            failed += 1
    await msg.edit(f"✅ **Broadcast
Done!**\n\n✅ Success: {success}\n❌
Failed: {failed}")


@bot.on_message(filters.command("stats") &
filters.user(ADMIN_IDS))
async def stats_cmd(client, message):
    total_users =
users_col.count_documents({})
    premium_users =
users_col.count_documents({"premium_until":
{"$gt": datetime.now()}})
    total_channels =
channels_col.count_documents({})
    active_channels =
channels_col.count_documents({"last_status"
: "active"})
```

```
    banned_channels =
channels_col.count_documents({"last_status"
: "banned"})
    await message.reply(
        f"📊 **BOT STATISTICS**\n\n"
        f"👥 **Total Users:**
{total_users:,}\n"
        f"💎 **Premium Users:**
{premium_users:,}\n"
        f"📡 **Total Channels:**
{total_channels:,}\n"
        f"✅ **Active:**
{active_channels:,}\n"
        f"❌ **Banned:**
{banned_channels:,}\n"
    )


@bot.on_callback_query()
async def callback_handler(client, cb):
    data = cb.data
    user_id = cb.from_user.id
    if data == "my_channels":
        channels = get_channels(user_id)
        limit = get_limit(user_id)
        if not channels:
            await cb.answer("Koi channel
add nahi kiya!", show_alert=True)
            return
        text = f"📋 **Monitored Channels
({len(channels)}/{limit}):**\n\n"
        for i, ch in enumerate(channels,
1):
            emoji = "✅" if
```

```
ch.get("last_status") == "active" else "❌"
            text += f"{i}. {emoji} **
{ch.get('title', ch['username'])}** —
@{ch['username']}\n"
        btn = [[InlineKeyboardButton("🔙
Back", callback_data="home")]]
        await cb.message.edit(text,
reply_markup=InlineKeyboardMarkup(btn))
    elif data == "how_to_add":
        await cb.message.edit(
            "➕ **Channel Add Kaise
Karo:**\n\n"
            "1. Command type karo:\n"
            "   `/add @yourchannel`\n\n"
            "2. Bot confirm karega aur
monitoring shuru!\n\n"
            "**Note:** Public channels
directly add ho jaate hain.",

reply_markup=InlineKeyboardMarkup([[InlineK
eyboardButton("🔙 Back",
callback_data="home")]])
        )
    elif data == "premium_menu":
        if is_premium(user_id):
            user = get_user(user_id)
            until =
user["premium_until"].strftime("%d %B %Y")
            await cb.answer(f"💎 Premium
active hai! Till {until}", show_alert=True)
            return
        btn = [
            [InlineKeyboardButton(f"💳 Buy
— ₹99/month",
```

```
url=f"https://t.me/{ADMIN_USERNAME}")],
            [InlineKeyboardButton("🔙
Back", callback_data="home")]
        ]
        await cb.message.edit(
            "💎 **PREMIUM —
₹99/month**\n\n"
            "✅ 1000+ channels monitor
karo\n"
            "✅ Real-time instant alerts\n"
            "✅ Message backup (last
100)\n"
            "✅ Priority support\n\n"
            f"Payment ke liye:
@{ADMIN_USERNAME}",

reply_markup=InlineKeyboardMarkup(btn)
        )
    elif data == "help_menu":
        btn = [[InlineKeyboardButton("🔙
Back", callback_data="home")]]
        await cb.message.edit(
            "📖 **Quick Help**\n\n"
            "`/add @ch` — Channel add
karo\n"
            "`/list` — Channels dekho\n"
            "`/remove @ch` — Remove karo\n"
            "`/status @ch` — Check karo\n"
            "`/keywords @ch w1,w2` —
Keywords\n"
            "`/premium` — Plan upgrade\n"
            "`/me` — Profile dekho",

reply_markup=InlineKeyboardMarkup(btn)
```

```
        )
    elif data == "home":
        name = cb.from_user.first_name
        await cb.message.edit(
            home_text(name, user_id),

reply_markup=home_buttons(user_id)
        )
    await cb.answer()


@bot.on_message(filters.channel &
~filters.scheduled)
async def channel_msg_listener(client,
message):
    try:
        chat = message.chat
        username = getattr(chat,
"username", None)
        if not username:
            return
        username = username.lower()
        text = (message.text or
message.caption or "").lower()
        if not text:
            return
        watchers = list(channels_col.find({
            "username": username,
            "keywords": {"$exists": True,
"$ne": []},
            "notify": True
        }))
        for watcher in watchers:
            keywords =
```

```
watcher.get("keywords", [])
            matched = [kw for kw in
keywords if kw in text]
            if not matched:
                continue
            uid = watcher["user_id"]
            kw_text = ", ".join([f"`{k}`"
for k in matched])
            preview = (message.text or
message.caption or "")[:400]
            if is_premium(uid):
                messages_col.insert_one({
                    "user_id": uid,
                    "channel": username,
                    "text": preview,
                    "msg_id": message.id,
                    "saved_at":
datetime.now(),
                })
            await bot.send_message(
                uid,
                f"🔔 **Keyword
Alert!**\n\n"
                f"📢 **Channel:**
@{username}\n"
                f"🔑 **Matched:**
{kw_text}\n\n"
                f"📝
**Message:**\n{preview}"
                f"{'...' if len(preview) >=
400 else ''}"
            )
    except Exception as e:
        log.error(f"Channel listener error:
```

```
{e}")


async def monitor_loop():
    await asyncio.sleep(30)
    log.info("Background monitor started!")
    while True:
        try:
            all_channels =
list(channels_col.find({}))
            log.info(f"Checking
{len(all_channels)} channels...")
            for ch in all_channels:
                try:
                    username =
ch["username"]
                    user_id = ch["user_id"]
                    old_status =
ch.get("last_status", "active")
                    info = await
fetch_channel_info(bot, username)
                    new_status = "active"
if info["alive"] else info.get("reason",
"unknown")

channels_col.update_one(
                        {"_id": ch["_id"]},
                        {"$set": {
                            "last_checked":
datetime.now(),
                            "last_status":
new_status,
                            "members":
info.get("members", ch.get("members", 0))
```

```
                        }}
                    )
                    if new_status ==
old_status:
                        await
asyncio.sleep(1)
                        continue
                    notify =
ch.get("notify", True)
                    if new_status ==
"banned" and notify:
                        await
bot.send_message(
                            user_id,
                            f"🚨 **URGENT:
Channel Ban Ho Gaya!**\n\n"
                            f"⛔
**Channel:** {ch.get('title', username)}\n"
                            f"🔗
**Username:** @{username}\n"
                            f"⏰ **Time:**
{datetime.now().strftime('%d %b %Y, %I:%M
%p')}\n\n"
                            f"😔 Channel ab
accessible nahi hai.\n\n"
                            f"💎
**Premium** lo aur message backup feature
use karo!\n"
                            f"Contact:
@{ADMIN_USERNAME}"
                        )
                    elif new_status ==
"deleted" and notify:
                        await
```

```
bot.send_message(
                            user_id,
                            f"⚠️ **Channel
Delete Ho Gaya!**\n\n"
                            f"🔗
@{username} ab exist nahi karta.\n"
                            f"Monitoring
list se hata diya gaya."
                        )

channels_col.delete_one({"_id": ch["_id"]})
                    elif new_status ==
"active" and old_status != "active" and
notify:
                        await
bot.send_message(
                            user_id,
                            f"✅ **Channel
Wapas Active Ho Gaya!**\n\n"
                            f"⛔
**Channel:** {ch.get('title', username)}\n"
                            f"🔗
**Username:** @{username}\n\n"
                            f"Channel ab
accessible hai!"
                        )
                    await asyncio.sleep(2)
                except FloodWait as e:
                    log.warning(f"FloodWait
{e.value}s")
                    await
asyncio.sleep(e.value)
                except Exception as e:
                    log.error(f"Error
```

```
checking @{ch.get('username')}: {e}")
            log.info(f"Monitor cycle done.
Next in {MONITOR_INTERVAL // 60} min.")
            await
asyncio.sleep(MONITOR_INTERVAL)
        except Exception as e:
            log.error(f"Monitor loop error:
{e}")
            await asyncio.sleep(60)


web = Flask("")


@web.route("/")
def home_route():
    try:
        total_users =
users_col.count_documents({})
        premium_users =
users_col.count_documents({"premium_until":
{"$gt": datetime.now()}})
        total_channels =
channels_col.count_documents({})
        active =
channels_col.count_documents({"last_status"
: "active"})
    except Exception:
        total_users = premium_users =
total_channels = active = "N/A"
    return (
        f"<h1>Channel Guardian Bot</h1>"
        f"<p>Status: LIVE</p>"
        f"<p>Users: {total_users}</p>"
```

```
        f"<p>Premium: {premium_users}</p>"
        f"<p>Channels: {total_channels}
</p>"
        f"<p>Active: {active}</p>"
    )


@web.route("/health")
def health():
    return {"status": "ok"}


def run_web():
    port = int(os.environ.get("PORT",
10000))
    web.run(host="0.0.0.0", port=port)


async def main():
    log.info("Starting Channel Guardian
Bot...")
    await bot.start()
    me = await bot.get_me()
    log.info(f"Bot started:
@{me.username}")
    asyncio.create_task(monitor_loop())
    await idle()
    await bot.stop()


if __name__ == "__main__":
    Thread(target=run_web,
daemon=True).start()
```

```
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main())
```